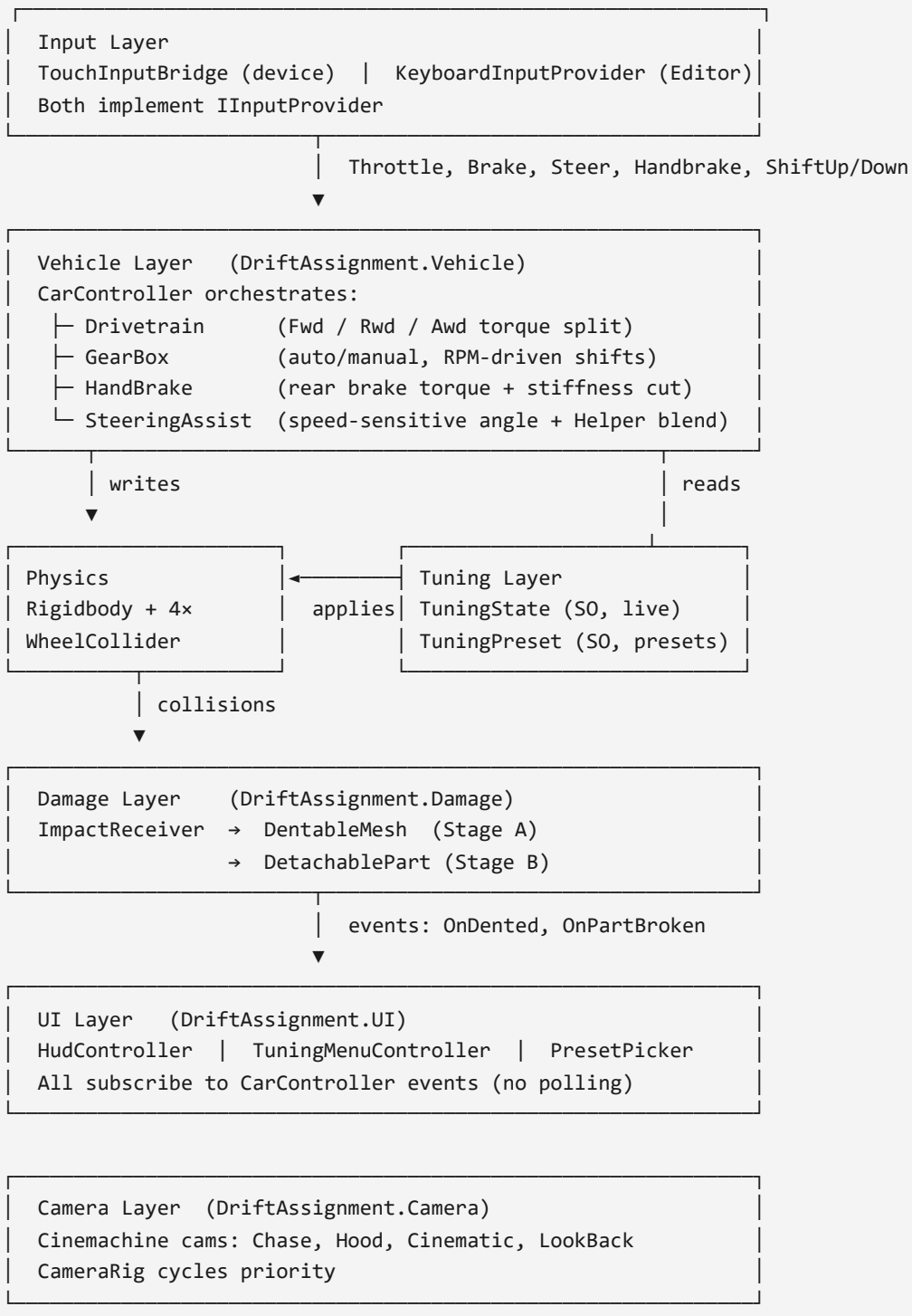
Drift Assignment — Architecture Document

> Companion to REQUIREMENTS.md. This document explains **how** the system is structured, why the boundaries are drawn where they are, and what the extensibility surface looks like.

---

1. High-Level System Layers



2. Module Responsibilities

Module (asmdef)	Responsibility	Key types	Depends on
`DriftAssignment.Core`	Shared enums, interfaces, math helpers	`IDamageable`, `DrivetrainMode`, `DamageStages`	(none)
`DriftAssignment.Input`	Input abstraction + implementations	`IInputProvider`, `TouchInputBridge`, `KeyboardInputProvider`	Core, Unity Input System
`DriftAssignment.Vehicle`	Car simulation	`CarController`, `Drivetrain`, `GearBox`, `HandBrake`, `SteeringAssist`, `WheelController`, `TuningState`, `TuningPreset`, `CarConfig`	Core, Input
`DriftAssignment.Damage`	Collision → dent / break	`ImpactReceiver`, `DentableMesh`, `DetachablePart`	Core
`DriftAssignment.UI`	HUD, tuning, presets	`HudController`, `TuningMenuController`, `PresetPicker`	Core, Vehicle (events only)
`DriftAssignment.Camera`	Camera rig	`CameraRig`	Cinemachine
`DriftAssignment.Character` *(stretch)*	Enter / exit driver	`CharacterEnterExit`, `ThirdPersonController`	Core, Vehicle

Dependency direction is enforced by asmdefs: **UI → Vehicle**, never reverse; **Vehicle → Input**, never reverse. Damage does not depend on UI or Input.

3. Data Flow

Per frame tick

```
 Input.Read()
   ↓
CarController.FixedUpdate()
   ↓
WheelColliders → Rigidbody
   ↓
fires events:
   ├─ OnRpmChanged   → HUD updates the needle
   ├─ OnGearChanged  → HUD updates the gear text
   └─ OnSpeedChanged → HUD updates KM/H
```

Tuning change (slider drag)

```
User drags slider
   ↓
```

```
TuningMenuController.OnValueChanged
   ↓
TuningState.SetSpringForce(v)
   ↓
fires OnTuningChanged
   ↓
CarController listens
   ↓
applies new value to WheelColliders
```

Preset apply

```
 PresetPicker.Select(presetSO)
   ↓
TuningPreset.CopyTo(TuningState)
   ↓
fires OnTuningChanged
   ↓
UI sliders re-read state and update
```

Collision

```
 PhysX OnCollisionEnter
   ↓
ImpactReceiver.HandleCollision(contacts)
   ├─ Route to nearest DentableMesh
   │      (Stage A: vertex displacement)
   └─ Add impulse to nearest DetachablePart._accumulatedImpact
          ↓
          if > breakThreshold → Detach() (Stage B)
```

---

4. Key Design Decisions

WheelCollider over custom raycast physics Well-documented, sufficient for sim-cade feel, no wheel-through-ground surprises after tuning substeps.

ScriptableObject for tuning & presets Designer-editable in Inspector, hot-swappable at runtime, testable in isolation. Presets are baked SOs (not JSON parsed) — zero-cost switching.

Interface-first input (`IInputProvider`) Touch, keyboard, and future replay/record providers all satisfy the same contract. `CarController` never touches `Input.GetAxis` or Touch APIs directly.

Damage split into Dent vs. Break controllers Single responsibility. Either can be stubbed independently for perf / A-B testing / low-end fallback.

Assembly Definitions from day one Compile speed (incremental scope) + architectural enforcement (dependency direction).

Event-driven UI, not polling HUD subscribes to `CarController` events (`OnGearChanged`, `OnRpmChanged`, `OnSpeedChanged`). No `Update()` on HUD widgets. Cheap and correct.

No gameplay singletons Dependencies injected through Inspector or via a lightweight `SceneServices` locator. Testable and swappable.

CoM offset via `Rigidbody.centerOfMass` Lower the center of mass to prevent rollover in drift, without artificial anti-roll bars.

---

5. Extensibility Points

Adding…	How	Cost
New drivetrain mode	Add enum member + case in `Drivetrain.ApplyTorque`	1 file
New car	New prefab + new `CarConfig` SO	0 code
New preset	Create `TuningPreset` SO — auto-appears in picker	0 code
New camera angle	Add virtual cam + register in `CameraRig`	1 file
Character enter / exit	Implement `IVehicleOccupant`; attach to character	1 new module
Replay input	Implement `IInputProvider` reading a recorded stream	1 new module

---

6. Trade-offs & Known Limits

- **WheelCollider is not deterministic across framerates** — acceptable for single-player, would be a blocker for lockstep multiplayer.
- **Runtime mesh deformation cost** may force a prefab-swap fallback on low-end Android; the fallback path is designed in from day one (see REQUIREMENTS §6).
- **No save / load system in scope** — tuning changes reset on scene reload. Explicit non-goal.
- **Single scene** — no async scene streaming for the desert road.

---

7. Testing Strategy

Edit-mode tests (fast, no PlayMode required)

- `TuningPreset.CopyTo` produces expected `TuningState` values
- `GearBox` shift-point math correct across auto and manual modes
- Damage impulse thresholding produces the correct `DamageStages` transitions

Play-mode tests (scene-driven)

- `CarController` spawns without null-refs on an empty scene
- A scripted collision produces one dent event and, at high impulse, one break event
- Preset switching produces a measurable change in wheel forward stiffness

Manual test matrix (per phase)

- Editor Play with keyboard: forward, reverse, steer, handbrake, camera cycle
- On-device APK smoke test: touch controls responsive, no ANR, no visible crash

---

8. Build & Run

Editor (dev loop)

1. Open `Assets/_Project/Scenes/Main.unity`
2. Press Play — keyboard controls active (WASD + Space handbrake)

Android APK

1. Player Settings: IL2CPP, ARM64, min API 24, .NET Standard 2.1
2. File → Build Settings → Android → Build
3. Deploy via `adb install -r drift.apk`

---

9. Related Documents

- REQUIREMENTS.md — scope, features, assets, style guide
- IMPLEMENTATION_LOG.md — phase-by-phase execution journal
- OPTIMIZATION.md — perf decisions with before/after impact
- CREDITS.md — third-party asset attribution